Credit Risk Prediction & MLOps Pipeline

SYSTEM ARCHITECTURE & GRADUATION PROJECT DOCUMENTATION

Date: Oct 18, 2026
Domain: MLOps / FinTech

1. PROJECT PLANNING & MANAGEMENT

> **1.1 Project Proposal Overview:** Build a robust, end-to-end production Machine Learning prediction API and MLOps pipeline for Credit Risk Classification. The system predicts credit default risk using the `credit_risk_dataset.csv` dataset to assist financial institutions in making automated, data-driven lending decisions.

Objectives

- **Production API:** Transform a credit risk model from an experimental Jupyter Notebook into a reproducible, version-controlled, production-ready API service.
- **MLOps Tracking:** Track, log, and register all machine learning experiments, parameters, metrics, and model artifacts using **MLflow**.
- **CI/CD & Automation:** Automate code testing, building, and containerized deployment using **Docker** and **GitHub Actions** CI/CD pipelines.

Project Scope

Covers exploratory data analysis (EDA), data preprocessing, feature engineering, model training/evaluation, experiment tracking with MLflow Model Registry, RESTful API development using FastAPI, Docker containerization, CI/CD automation, and cloud deployment.

1.2 PROJECT PLAN & TIMELINE (SEQUENTIAL HANDOVER SCHEDULE)

Team Member & Role	Task & Scope	Start Date	End Date
1. Nagwa Mohammed *Data Preparation & EDA*	EDA, missing value imputation, outlier handling, feature correlation analysis, and dataset splitting (Train/Val/Test).	**Oct 19, 2026**	**Oct 30, 2026**
2. Jana Emara *Model Development*	Baseline modeling, algorithm selection (Logistic Regression, Random Forest), feature engineering, and ROC-AUC/F1 evaluation.	**Oct 31, 2026**	**Nov 10, 2026**
3. Dina Abdelmnhem *Experiment Tracking & Registry*	MLflow setup, parameter/metric logging, run comparison, and optimal model registration in MLflow Model Registry.	**Nov 11, 2026**	**Nov 18, 2026**
4. Abir Mohamed *Prediction API*	Retrieve registered model, develop FastAPI application, build `POST /predict` endpoint, and handle input validation.	**Nov 19, 2026**	**Nov 24, 2026**
5. Hager Sameh *Docker & CI/CD*	Containerize API with multi-stage `Dockerfile`, configure `GitHub Actions` for unit testing, and automate build checks.	**Nov 25, 2026**	**Nov 28, 2026**
6. Mariem Yousry *Deployment & Integration*	Deploy containerized service to cloud (Hugging Face/Azure), configure secrets, conduct system tests, and finalize docs.	**Nov 29, 2026**	**Dec 01, 2026**

1.3 DETAILED TASK ASSIGNMENT & ROLE BREAKDOWN

Nagwa Mohammed — Data Prep & EDA

Responsible for raw data inspection on `credit_risk_dataset.csv`, handling missing values, identifying outliers, analyzing feature correlations with default risk, and generating stratified splits.

Jana Emara — Model Development

Establish baseline benchmark models, train classification algorithms (Logistic Regression, Random Forest), implement preprocessing transformers, and evaluate ROC-AUC and F1-scores.

Dina Abdelmnhem — Experiment Tracking

Integrate MLflow tracking server, log hyperparameters, metrics, and artifacts across experiment runs, manage model versioning, and promote production candidates to Registry.

Abir Mohamed — Prediction API

Construct asynchronous REST API with FastAPI, expose `POST /predict` endpoint, validate JSON schemas using Pydantic, load ML pipelines, and output probability metrics.

Hager Sameh — Docker & CI/CD

Package application using Docker, optimize image size via multi-stage builds, and construct GitHub Actions pipelines to execute automated unit testing on code commits.

Mariem Yousry — Deployment & Integration

Host containerized service on cloud platforms (Hugging Face Spaces / Azure), configure environment variables, perform end-to-end integration testing, and author final system README.

1.4 RISK ASSESSMENT & MITIGATION PLAN

- **Risk 1: Data and Concept Drift**
  Mitigation: Leverage MLflow Model Registry for systematic versioning and configure periodic model evaluation/retraining pipelines.
- **Risk 2: High Latency during API Inferences**
  Mitigation: Streamline preprocessing pipelines and run lightweight ASGI servers (Uvicorn) within optimized Docker containers.
- **Risk 3: CI/CD Pipeline Failures due to Dependency Mismatches**
  Mitigation: Pin exact library versions in `requirements.txt` and mandate unit test suites prior to PR approval.

1.5 KEY PERFORMANCE INDICATORS (KPIS)

> 0.85	< 200 ms	99.5%	< 5 min
ROC-AUC / F1	API LATENCY	SERVICE UPTIME	CI/CD BUILD

2. LITERATURE REVIEW

2.1 Analysis of Existing & Similar Solutions

- **Traditional Credit Scoring Models:** Rely on simple statistical models like standard Logistic Regression. While highly interpretable, they lack capacity for non-linear feature interactions and lack modern MLOps automation.
- **Standalone ML Models without API/Containerization:** Utilize strong algorithms (Random Forest, XGBoost) but remain trapped inside Jupyter Notebooks without web serving capabilities or containerized deployment pipelines.

2.2 Comparative Advantage & Value Addition

Our project bridges the gap between ML experimentation and production engineering by providing:

- **Centralized Experimentation:** Full auditability of model parameters, metrics, and code versions via **MLflow**.
- **Production-Ready Inferences:** Real-time risk scoring delivered via asynchronous **FastAPI** microservices.
- **Automated MLOps Lifecycle:** Environment parity ensured by **Docker** containers and continuous quality checks automated through **GitHub Actions**.

2.3 Technical References

- Scikit-Learn & Pandas Official Technical Documentation.
- MLflow Tracking & Model Registry Guides.
- FastAPI & OpenAPI Specification Standards.
- Kaggle Credit Risk Dataset Repositories & Industry Benchmarks.

3. REQUIREMENTS GATHERING

3.1 Stakeholder Analysis

Credit & Risk Assessment Teams

Require accurate, real-time risk scores to evaluate loan applicants and minimize default rates.

Customer Service Representatives

Need fast, understandable decision outputs to streamline customer loan approval workflows.

App Developers & Engineers

Require well-documented API endpoints (Swagger/OpenAPI) for seamless frontend integration.

MLOps & DevOps Engineers

Require reproducible builds, containerized execution, and version-controlled model artifacts.

3.2 User Stories & System Use Case

> **User Story:** *"As a Credit Officer, I want to submit an applicant's financial profile into the system so that I can receive an instant risk rating and default probability score to decide on loan approval."*

1. Client application sends a JSON payload containing applicant attributes to the API.
2. The `POST /predict` endpoint validates schema compliance via Pydantic.
3. Input data passes through the saved preprocessing pipeline for transformation.
4. The registered model computes and returns default probability and risk classification.

3.3 Functional Requirements

- **Data & Modeling:** Preprocess raw inputs, impute missing values, scale numerical features, and execute ML predictions.
- **Experimentation:** Log parameters, metrics, and models to MLflow; register production-ready models.
- **RESTful API:** Expose a `POST /predict` endpoint with input schema validation and structured JSON outputs.
- **Containerization & CI/CD:** Package service with Docker and automate testing using GitHub Actions.

3.4 Non-Functional Requirements

- **Performance:** Real-time prediction responses (<200 ms latency).
- **Security & Reliability:** Secure environment configuration, safe data handling, and 99.5% service availability.
- **Reproducibility:** Consistent model execution across local and cloud setups via Docker and MLflow.